# SpendWise - Documentation

**Version:** 1.0.0

**Developer/Author:** Arslan Ahmed

**Date:** January 29, 2026

**Table of Contents**

# 1. Introduction

**SpendWise** is a modern, cross-platform personal finance application designed to help users track expenses, manage wallets, and visualize their financial health. Built with a "mobile-first" philosophy, it leverages **React Native** for a seamless user experience across Android and iOS, **Firebase** for real-time data synchronization and authentication, and **Cloudinary** for optimized media management.

**Key Features**

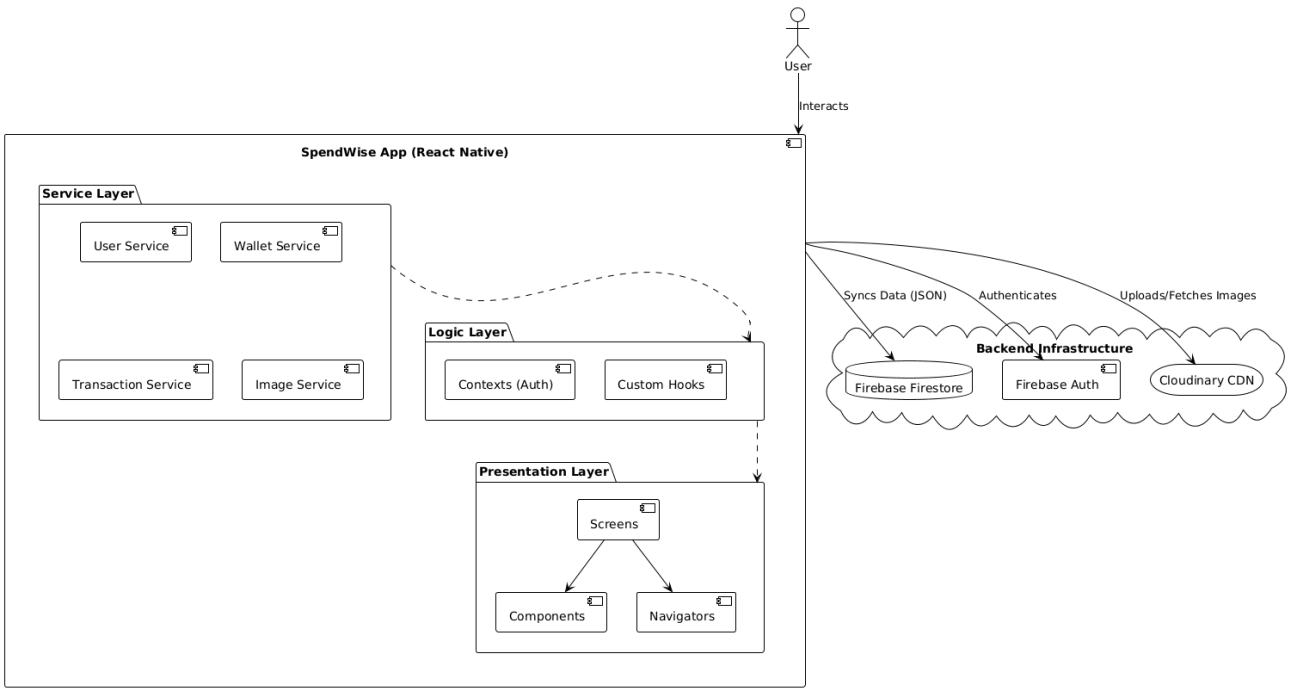
- **Multi-Wallet Management:** Track distinct accounts (e.g., Savings, Cash, Credit).
- **Transaction Tracking:** Record income and expenses with categorical breakdowns.
- **Visual Analytics:** Interactive charts for weekly, monthly, and yearly financial trends.
- **Cloud Synchronization:** Real-time data sync across devices via Firestore.
- **Secure Authentication:** robust user management powered by Firebase Auth.

# 2. System Architecture

SpendWise follows a **Service-Oriented Architecture (SOA)** within a serverless environment. The application is structured into distinct layers to ensure separation of concerns, maintainability, and scalability.

## 2.1 High-Level Architecture Diagram

The following diagram illustrates the data flow between the Client (React Native), the Service Layer, and the Backend providers (Firebase/Cloudinary).



## 2.2 Directory Structure & Responsibilities

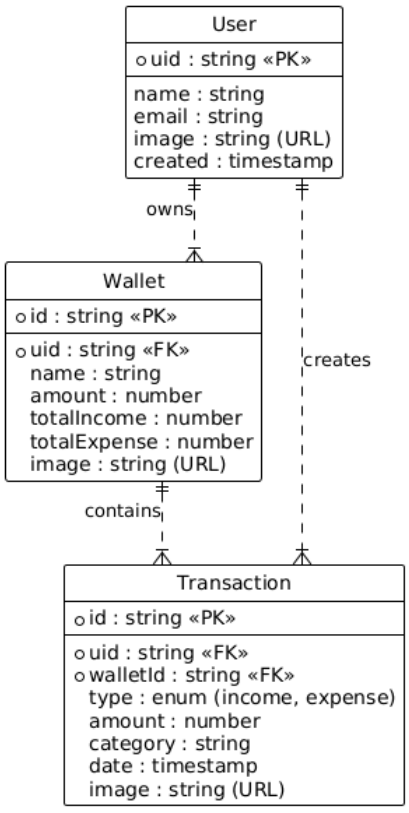
The codebase is organized to support scalability and feature isolation:

| Directory | Responsibility |
|---|---|
| `/app` | **Presentation Layer:** Handles UI and routing (Expo Router). Contains screens for Auth, Tabs, and Modals. |
| `/components` | **UI Library:** Reusable, stateless UI elements (Buttons, Inputs, Charts). |
| `/services` | **Business Logic:** API calls and data transformation. Isolates the UI from backend complexity. |
| `/contexts` | **State Management:** Global state (e.g., AuthContext) accessible throughout the app. |
| `/hooks` | **Custom Logic:** Reusable React hooks, primarily useFetchData for Firestore listeners. |
| `/constants` | **Configuration:** Design tokens (Theme, Colors) and static app data. |

# 3. Database Schema

SpendWise utilizes **Google Cloud Firestore**, a NoSQL document database. The data model is designed for high read performance, utilizing collections for Users, Wallets, and Transactions.

## 3.1 Entity Relationship Diagram (ERD)



### 3.2 Collection Details

#### Users (`users/{uid}`)

Stores user profile data.

- **Security:** Users can only read/write their own documents.
- **Indexes:** Primary index on uid.

#### Wallets (`wallets/{walletId}`)

Represents financial accounts.

- **Key Fields:** amount (current balance), totalIncome, totalExpense.
- **Logic:** Balance is recalculated whenever a contained transaction is added, modified, or deleted.

#### Transactions (`transactions/{transactionId}`)

Individual financial records.

- **Key Fields:** type (Income/Expense), category, date.
- **Indexing:** Composite indexes are required for filtering by Date, Type, and Category simultaneously.

## 4. API & Services Layer

The application interacts with the backend through specialized service modules found in the `/services` directory. All services return a standardized ResponseType object: `{ success: boolean, data?: any, msg?: string }`.

### 4.1 Transaction Service (`transactionService.ts`)

Handles the complex logic of moving money.

- **`createOrUpdateTransaction`**: This is a transactional operation. If a new transaction is created:
    1. The transaction document is saved to Firestore.
    2. The associated Wallet's `amount`, `totalIncome`, and `totalExpense` are updated atomically.
- **`deleteTransaction`**: Reverses the financial impact on the wallet before deleting the transaction record.

### 4.2 Image Service (`imageService.ts`)

Manages media assets via Cloudinary.

- **`uploadFileToCloudinary`**: Accepts a file URI, compresses it, and uploads it to specific folders (`users/`, `wallets/`, etc.) using an unsigned upload preset. Returns a secure HTTPS URL for database storage.

### 4.3 Data Fetching Hook (`useFetchData`)

A custom hook that abstracts Firestore real-time listeners. It handles:

- Loading states (`loading: true/false`).
- Error handling.
- Real-time updates (UI refreshes automatically when data changes on the backend).

## 5. Frontend Component Library

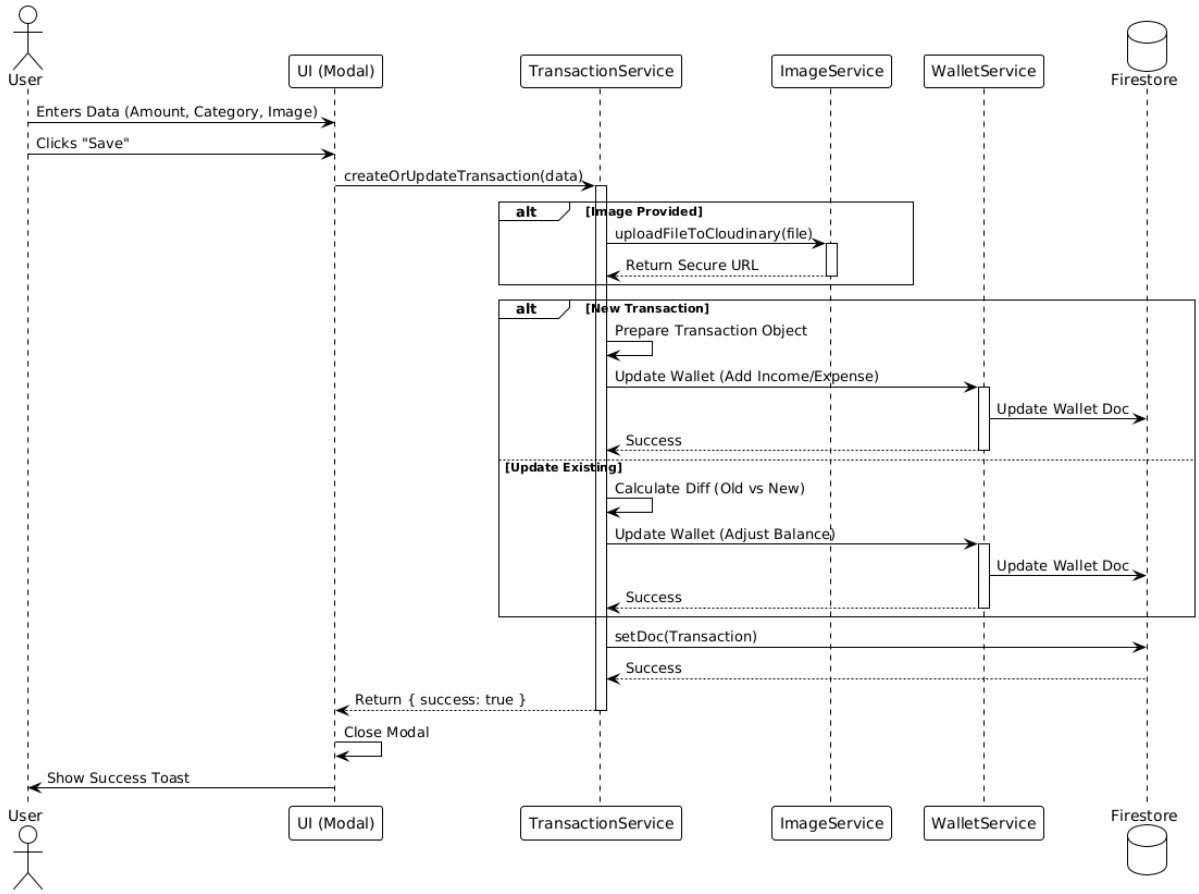
SpendWise utilizes a custom design system to ensure consistency.

### 5.1 Core Components

- **ScreenWrapper:** A Higher-Order Component (HOC) that handles Safe Area Views across different devices (notches, dynamic islands).
- **Typo:** A unified typography component that enforces font families, weights, and sizing scaling.
- **Button/Input:** Custom form elements with built-in loading states and validation styling.

### 5.2 Transaction Flow

The following sequence diagram demonstrates the flow when a user adds a new expense.




# 6. Setup & Configuration

## 6.1 Prerequisites

- **Node.js**: v18.x or later
- **JDK**: Version 21
- **Android SDK**: API Level 34

## 6.2 Installation

### 6.2.1 Clone the Repository:

```
git clone https://github.com/mearslanahmed/SpendWise.git

cd SpendWise
```

### 6.2.2 Install Dependencies:

```
npm install
```

### 6.2.3 Environment Setup:

Create a .env file in the root directory and populate it with your API keys:

```
EXPO_PUBLIC_FIREBASE_API_KEY=your_key

EXPO_PUBLIC_FIREBASE_PROJECT_ID=your_id
```

```
EXPO_PUBLIC_CLOUDINARY_CLOUD_NAME=your_cloud_name
```

### 6.3 Running the App

- **Android:** npm run android
- **iOS:** npm run ios (macOS only)
- **Web:** npm run web

## 7. Deployment Guide

### 7.1 Build Process

SpendWise uses **EAS (Expo Application Services)** and standard Gradle builds for production.

**Android Release Build:**

1. Increment the `versionCode` in `app.json`.
2. Run the Gradle bundle command:

```
cd android
./gradlew bundleRelease
```

3. The output `.aab` file will be located in `android/app/build/outputs/bundle/release/`.

### 7.2 Security Rules

Before deployment, ensure Firestore rules are enforced to prevent unauthorized access:

```
service cloud.firestore {
  match /databases/{database}/documents {
    // Users can only access their own data
    match /{collection}/{document=**} {
      allow read, write: if request.auth.uid ==
resource.data.uid;
    }
  }
}
```